2

Databases 1 · Week 2

The relational model: ER modelling

Hierarchical = tree of parents; network = graph of pointers. Both make programs navigate.

- Codd (1970): data as relations, linked by values, queried declaratively.
- It became practical in the early 1980s: faster hardware, query optimisers, and SQL.
- Data independence: change storage without rewriting programs.

> **Think of it like this.** A navigational database is like giving a friend turn-by-turn directions you memorised; if one street closes, the directions are useless. A relational database is like giving them the address: the navigation app (the optimiser) works out the route, and a closed street only changes the route, not the address.

Relation = table, tuple = row, attribute = column.

- Domain = the allowed values of an attribute.
- Degree = number of attributes; cardinality = number of tuples.
- No duplicate tuples, no meaningful order, one atomic value per cell.
- NULL marks a missing value; test it with IS NULL.

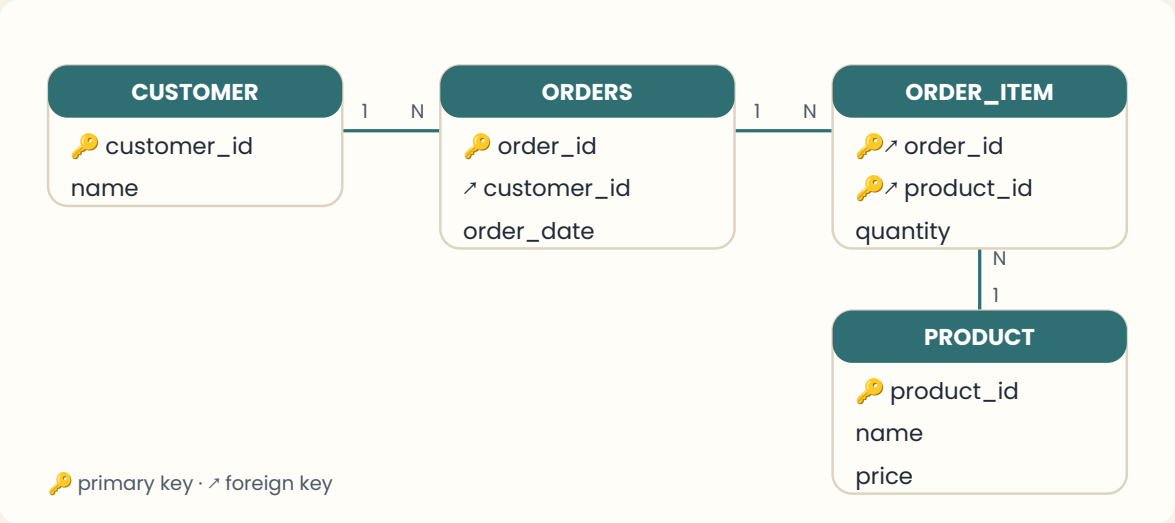
> **Think of it like this.** A relation is like the playlist screen of a music app: the column headings (title, artist, length) are the schema, each line is a tuple, the length of a song must fit its domain (no negative minutes), and sorting the list by title does not change which songs are in it.

Superkey: unique, possibly with extra attributes.

- Candidate key: a minimal superkey. There may be several.
- Primary key: the chosen candidate; unique, never NULL, minimal (irreducible), and stable. Alternate keys get UNIQUE.
- Composite key: unique only as a combination.
- Foreign key: a copy of another relation's primary key.

> **Think of it like this.** Think of ways to find a friend at a festival. "Wearing a red cap" is not a key (many people do). "Phone number + red cap" is a superkey: unique, but the cap is extra. "Phone number" alone is a candidate key, and so is "ticket number". The one you write in your contacts is the primary key.

Entity integrity: a primary key is never NULL.



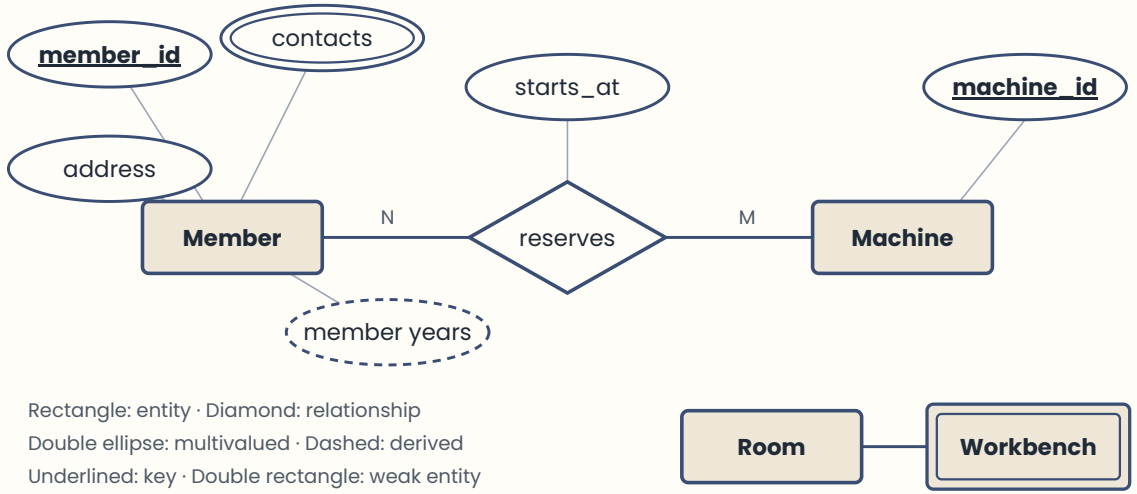
> **Think of it like this.** A foreign key is like a coat-check ticket. Referential integrity means every ticket in someone's hand matches a coat on the rack. RESTRICT: the cloakroom will not throw away a coat while its ticket is out. CASCADE: if the coat goes, the ticket is torn up too. SET NULL: the ticket stays, but now it points to nothing.

1:N: foreign key on the "many" side.

- 1:1: foreign key on one side, plus UNIQUE.
- N:M: junction table with two foreign keys forming a composite primary key.
- Attributes of the pair go in the junction table.

> **Think of it like this.** 1:N is a parking lot: each car has one ticket stub with the lot number, and the lot does not keep a list of cars. N:M is a dance class sign-up sheet: neither the dancers nor the classes keep the list; the sheet itself has one line per (dancer, class) pair.

Requirements → conceptual (ER) → logical (relations) → physical (SQL).



Think of it like this. An ER diagram is like a family tree drawn for a notary: boxes for people (entities), lines labelled "married to" or "child of" (relationships), notes on each person (attributes), and rules such as "every child has exactly two parents" (participation and cardinality).

Specialisation: top-down; generalisation: bottom-up. Both give an ISA hierarchy.

- Subtypes inherit all attributes and relationships of the supertype.
- Disjoint (d) vs overlapping (o); total (double line) vs partial (single line).
- Map with a table per entity, per subtype (total and disjoint only), or one table with a type column.

> **Think of it like this.** A hierarchy is like a menu: every dish has a name and a price (supertype), but pizzas also have a crust and drinks a size (subtypes). A combo that is both a pizza and a drink would make the menu overlapping; a menu where every item must be a pizza or a drink would be total.

Chen: rectangles, diamonds, ovals; 1/N on lines; double line = total.

- Crow's foot: boxes with attributes; | one, o zero, fork many.
- Inner mark = minimum, outer mark = maximum.
- Read each direction from the symbol at the far end.

> **Think of it like this.** Chen and crow's foot are like a hand-drawn map and a subway map of the same city: the first shows every park and building, the second simplifies to lines and stops so you can find your route quickly. Same city, different purpose.

Context → requirements → entities → attributes → relationships.

- Entities are nouns the organisation remembers many of, each identifiable.
- Totals such as monthly income are derived: compute them, do not store them.
- Tick every numbered requirement off against the final diagram.

> **Think of it like this.** The method is like packing for a trip: first decide where you are going (context), write the list (requirements), pick the bags (entities), fill them (attributes), and note which items go together (relationships). Checking the list at the door is the final pass over the requirements.

Order: strong → weak → 1:1 → 1:N → N:M → multivalued → n-ary.

- Weak entity key = owner key + partial key.
- Total participation on the "many" side becomes NOT NULL.
- Check the result against the requirements, sentence by sentence.

> **Think of it like this.** Mapping is like following a recipe card after you have bought all the ingredients: the creative part (choosing the dish, the ER design) is done, and now each step is fixed: chop, fry, season, plate. Skipping a step is how you end up with a missing table.

Money: DECIMAL/NUMERIC(p,s), never FLOAT.

- Auto ids: AUTO_INCREMENT (MySQL) vs SERIAL/BIGSERIAL (PostgreSQL).
- DATETIME (MySQL) vs TIMESTAMP/TIMESTAMPTZ (PostgreSQL); BOOLEAN is TINYINT(1) in MySQL.
- Change a type: MODIFY (MySQL) vs ALTER COLUMN … TYPE (PostgreSQL).
- DELETE rows, TRUNCATE all rows, DROP the table.

> **Think of it like this.** DELETE, TRUNCATE, and DROP are like cleaning a bookshelf: DELETE takes out the books you choose one by one, TRUNCATE sweeps every book off at once but leaves the shelf, and DROP carries the shelf itself out of the room.

INSERT with a column list; many rows in one statement; parents first.

- SELECT columns AS alias FROM t WHERE … ORDER BY … LIMIT n.
- UPDATE … SET a = 1, b = 2 WHERE …; DELETE FROM … WHERE ….
- Test the WHERE with a SELECT before changing data.

> **Think of it like this.** DML is like editing a contacts app: INSERT adds people (or imports a whole list at once), SELECT searches and sorts them, UPDATE edits one person's details, and DELETE removes a contact. Pressing "delete all" by mistake is what a missing WHERE does.

Step by step

From pointers to tables

1. Hierarchical and network databases run business systems
2. Codd publishes the relational model (1970)
3. Research prototypes prove it works and SQL is born
4. Commercial relational systems become viable (early 1980s)
5. The relational model becomes the default for business data

Check yourself

1. What was the main weakness of hierarchical and network databases?

A) Programs had to navigate stored paths, so new questions and structure changes meant new code B) They could not store text C) They were always slower than relational systems

2. LISTENER(listener_id, email, display_name). listener_id and email are each unique. Which set is a superkey but NOT a candidate key?

A) {email, display_name} B) {email} C) {display_name}

3. A streaming service numbers tracks 1, 2, 3… inside each album. TRACK(album_id, track_no, song_id). What is the primary key?

A) (album_id, track_no) B) track_no C) album_id

Answers are in the speaker notes.

Practise next

Labs

- Build a streaming schema with 1:N and N:M
- Integrity rules in action
- From a real scenario to a logical model
- Model a community garden from its description

On the site

- Design Studio
- SQL Lab